

16-bit CPU — Instruction Set Reference

Custom single-cycle CPU project — ISA documentation

Instruction Word Format

Every instruction is 16 bits wide, split into four 4-bit fields:

Bits	[15:12]	[11:8]	[7:4]	[3:0]
Field	Opcode	mode_bits (MOV / Branch / Shift groups) or Rd (ALU-type groups)	Ra (source A) or Rd (MOV/Shift groups)	Rb (source B) or 4-bit immediate

For ADD/SUB/AND/OR/NOT/XOR/MUL/MULS: Rd = bits [11:8], Ra = bits [7:4], Rb = bits [3:0]. For MOV and Shift groups: bits [11:8] hold the sub-mode instead, and the destination register moves down to bits [7:4].

Top-Level Opcode Map

Opcode	Instruction
0000	RST
0001	MOV (group — see sub-modes below)
0010	Branch / Compare (group — see sub-modes below)
0011	Shift (group — see sub-modes below)
0100	ADD Rd, Ra, Rb
0101	ADD Rd, Ra, #imm
0110	SUB Rd, Ra, Rb
0111	SUB Rd, Ra, #imm
1000	AND
1001	OR
1010	NOT
1011	XOR
1100	MUL
1101	MULS
1110	NOP
1111	HLT

MOV Sub-modes (mode_bits, opcode = 0001)

mode_bits	Instruction	Notes
0000	MOV Rd, Rs (register to register)	OPEN ISSUE: currently routes through the ALU with the default alu_op (ADD)
0011	MOV Rd, #imm	Confirmed against working test program.

mode_bits	Instruction	Notes
0100	MOV [Ra], Rb (store to memory)	Address = Ra, data = Rb. Confirmed.
1000	MOV Rd, [Ra] (load from memory)	Address = Ra (bit-field position may differ — see final_CPU wiring). Confirmed.
0001–0010, 0101–1111 (except above)	Not yet defined	Left for future expansion.

Shift Sub-modes (mode_bits, opcode = 0011)

mode_bits	Instruction	Notes
0000	Rshift Ra, Rb	SHR
0001	Rshift Ra, #imm	SHR, alu_src=1
0010	Lshift Ra, Rb	SHL
0011	Lshift Ra, #imm	SHL, alu_src=1
0100	Arith-right Ra, Rb	ASHR
0101	Arith-right Ra, #imm	ASHR, alu_src=1
0110	ROR Ra, Rb	Rotate right
0111	ROR Ra, #imm	alu_src=1
1000	ROL Ra, Rb	Rotate left
1001	ROL Ra, #imm	alu_src=1

Destination register for the Shift group is bits [7:4] (same field as the source register — result overwrites the source).

Branch / Compare Sub-modes (mode_bits, opcode = 0010)

mode_bits	Mnemonic	Condition checked
0000	B	Unconditional
0001	BZ	zero == 1
0010	BNZ	zero == 0
0011	BC	carry == 1
0100	BNC	carry == 0
0101	BV	overflow == 1
0110	BNV	overflow == 0
0111	—	unused
1000	BEQ	eq == 1
1001	BNE	eq == 0
1010	BGT	gt == 1

mode_bits	Mnemonic	Condition checked
1011	BLT	lt == 1
1100	BGE	gt == 1 OR eq == 1
1101	BLE	lt == 1 OR eq == 1
1110	CMPS	signed compare — sets alu_op, branch forced to 0
1111	CMP	unsigned compare — sets alu_op, branch forced to 0

Flags (zero/carry/sign/overflow/eq/gt/lt) are latched into a dedicated flag register on ADD, SUB, CMP, or CMPS — not read live off the ALU — so a branch correctly sees the result of the most recent flag-setting instruction rather than whatever the ALU happens to be computing that cycle.

ALU Operation Encoding (alu_op)

alu_op	Operation
0	ADD
1	SUB
2	MUL (unsigned)
3	MULS (signed / Booth)
4	ASHR
5	SHL
6	SHR
7	ROL
8	ROR
9	CMP (unsigned) — outs = eq_u
10	CMPS (signed) — outs = eq_s
11	AND
12	OR
13	NOT
14	XOR
15	MOV pass-through (outs = b)

OPEN ISSUE: this numbering was introduced in a later revision of the ALU module. main_decoder.v must emit these exact values — earlier decoder versions used a different numbering (e.g. AND=2, MUL=6, CMP=13). If the decoder was not updated to match, ALU-routed instructions (and the flag-latch condition, which checks for specific alu_op values) will silently misbehave. Verify decoder and ALU agree, opcode by opcode, before trusting results.

Flags

Flag	Definition
Z (zero)	Set when the ALU result is zero (for ADD/SUB). NOTE: for CMP/CMPS, outs holds the eq bit itself rather than an
C (carry)	Set from the adder/subtractor's carry-out on ADD/SUB only. 0 for all other operations.
S (sign)	MSB of the ALU result.
V (overflow)	Signed overflow, computed only for ADD/SUB from operand sign bits vs. result sign bit.
eq / gt / lt	Produced by cmp0 (unsigned) or cmps (signed), selected by whether the current op is CMPS. Always live; only m

Known Open Issues (as of this document)

1. MOV Rd,Rs (reg-to-reg copy) incorrectly routes through the ALU's default ADD path instead of a pure copy.
2. alu_op numbering was changed in the ALU rewrite; main_decoder.v must be re-verified against the new table above.
3. Zero-flag / eq-flag computation for CMP and CMPS needs a special case rather than the generic (outs==0) check.
4. 4-bit immediate sign-extension behavior (sign-extended vs. zero-extended) has not been empirically confirmed against the sign_extend module.
5. SUB's carry/borrow output polarity convention has not been explicitly confirmed by a hand-traced test case.
6. MOV sub-modes beyond #imm / load / store are not yet defined.
7. Branch target addressing mode (absolute vs. PC-relative) and the exact bit range used for the target are implemented as absolute, low byte of the instruction — confirm this matches the final design intent.